

PROPRIETARY
&
CONFIDENTIAL

— 機密文件

API SECURITY AUDIT

業
級
API
安
全
稽
核
報

專案名稱

(Project): vAmPI
Scan

目標位址 (Base
URL): http://
vampi:5000

稽核編號 (Audit
ID): #26

生成日期
(Date): 2026-02-28
09:51:59

1. 執行摘要 (Executive Summary)

本報告記錄了對 **vAmPI Scan** (<http://vampi:5000>) 進行自動化 API 安全稽核的完整發現。本次稽核遵循 **OWASP API Security Top 10 (2023)** 標準，涵蓋授權缺失、資料洩漏、安全設定錯誤等風險類別，並透過 **AI (GPT) 語意分析** 生成個別化修復方案，提供可立即執行的技術建議。

本次評測識別 **11** 個安全發現。所有高危 (High) 項目建議 **立即進行修補**，中低危項目請納入下次開發衝刺的修復計劃。

風險類別	嚴重等級	數量	建議優先級
Critical / High-Risk Findings	高 (High)	11	立即修補 (Immediate)
Medium-Risk Findings	中 (Medium)	0	計劃修補 (Planned < 30d)
Low-Risk / Informational	低 (Low)	0	例行追蹤 (Routine)
Total Findings	—	11	—

* 評等依據：CVSS 可利用性分數 × 業務資料影響評估。AI 分析由 OpenAI GPT 模型提供。

2. 稽核發現與 AI 修復方案 (Findings & Remediation)

GET /

A | 業務影響評估 (BUSINESS IMPACT)

攻擊者可以用普通用戶身份呼叫管理功能，導致帳號提權、資料竄改或系統控制。對企業造成法規違規（如 GDPR、PCI-DSS）和聲譽損失。

B | 技術根因分析 (ROOT CAUSE & TECHNICAL ANALYSIS)

根因：API 端點未在伺服器端驗證調用者的角色權限，僅依賴前端隱藏按鈕或 UI 限制，未實施真正的後端授權檢查。

技術分析：掃描器使用低權限 token 成功呼叫應僅限管理員的端點，伺服器返回 200 OK 而非 403 Forbidden，確認後端缺少角色驗證邏輯。

偵測證據 (Evidence)：

Status: 200

C | AI 修復方案 (REMEDIATION STEPS)

AI

- 在每個敏感端點加入角色檢查 decorator
- 建立集中式權限中介層 (middleware)
- 採用 RBAC（角色存取控制）框架
- 定期進行授權邏輯的 Code Review
- 在 CI/CD 中加入自動化授權測試

D | 修復程式碼範例 (CODE EXAMPLE)

▸ Recommended Implementation

```
# FastAPI 範例：正確的角色檢查
from fastapi import Depends, HTTPException

def require_admin(current_user = Depends(get_current_user)):
    if current_user.role != 'admin':
        raise HTTPException(status_code=403, detail='Admin only')
    return current_user

@router.get('/admin/users')
def get_users(admin = Depends(require_admin)):
    return db.query(User).all()
```

E | 合規框架映射 (COMPLIANCE MAPPING)

OWASP API1:2023 (Broken Object Level Authorization) | PCI-DSS Req 6.4.1 (Protect against unauthorized access) | NIST SP 800-53 AC-3 (Access Enforcement)

GET /

A | 業務影響評估 (BUSINESS IMPACT)

攻擊者可以用普通用戶身份呼叫管理功能，導致帳號提權、資料竄改或系統控制。對企業造成法規違規（如 GDPR、PCI-DSS）和聲譽損失。

B | 技術根因分析 (ROOT CAUSE & TECHNICAL ANALYSIS)

根因：API 端點未在伺服器端驗證調用者的角色權限，僅依賴前端隱藏按鈕或 UI 限制，未實施真正的後端授權檢查。

技術分析：掃描器使用低權限 token 成功呼叫應僅限管理員的端點，伺服器返回 200 OK 而非 403 Forbidden，確認後端缺少角色驗證邏輯。

偵測證據 (Evidence)：

Status: 200

C | AI 修復方案 (REMEDIATION STEPS)

AI

- 在每個敏感端點加入角色檢查 decorator
- 建立集中式權限中介層 (middleware)
- 採用 RBAC（角色存取控制）框架
- 定期進行授權邏輯的 Code Review
- 在 CI/CD 中加入自動化授權測試

D | 修復程式碼範例 (CODE EXAMPLE)

▸ Recommended Implementation

```
# FastAPI 範例：正確的角色檢查
from fastapi import Depends, HTTPException

def require_admin(current_user = Depends(get_current_user)):
    if current_user.role != 'admin':
        raise HTTPException(status_code=403, detail='Admin only')
    return current_user

@router.get('/admin/users')
def get_users(admin = Depends(require_admin)):
    return db.query(User).all()
```

E | 合規框架映射 (COMPLIANCE MAPPING)

OWASP API1:2023 (Broken Object Level Authorization) | PCI-DSS Req 6.4.1 (Protect against unauthorized access) | NIST SP 800-53 AC-3 (Access Enforcement)

GET /books/v1

A | 業務影響評估 (BUSINESS IMPACT)

攻擊者可以用普通用戶身份呼叫管理功能，導致帳號提權、資料竄改或系統控制。對企業造成法規違規（如 GDPR、PCI-DSS）和聲譽損失。

B | 技術根因分析 (ROOT CAUSE & TECHNICAL ANALYSIS)

根因：API 端點未在伺服器端驗證調用者的角色權限，僅依賴前端隱藏按鈕或 UI 限制，未實施真正的後端授權檢查。

技術分析：掃描器使用低權限 token 成功呼叫應僅限管理員的端點，伺服器返回 200 OK 而非 403 Forbidden，確認後端缺少角色驗證邏輯。

偵測證據 (Evidence)：

Status: 200

C | AI 修復方案 (REMEDIATION STEPS) AI

- 1. 在每個敏感端點加入角色檢查 decorator
- 2. 建立集中式權限中介層 (middleware)
- 3. 採用 RBAC（角色存取控制）框架
- 4. 定期進行授權邏輯的 Code Review
- 5. 在 CI/CD 中加入自動化授權測試

D | 修復程式碼範例 (CODE EXAMPLE)

▸ Recommended Implementation

```
# FastAPI 範例：正確的角色檢查
from fastapi import Depends, HTTPException

def require_admin(current_user = Depends(get_current_user)):
    if current_user.role != 'admin':
        raise HTTPException(status_code=403, detail='Admin only')
    return current_user

@router.get('/admin/users')
def get_users(admin = Depends(require_admin)):
    return db.query(User).all()
```

E | 合規框架映射 (COMPLIANCE MAPPING)

OWASP API1:2023 (Broken Object Level Authorization) | PCI-DSS Req 6.4.1 (Protect against unauthorized access) | NIST SP 800-53 AC-3 (Access Enforcement)

GET /books/v1

A | 業務影響評估 (BUSINESS IMPACT)

攻擊者可以用普通用戶身份呼叫管理功能，導致帳號提權、資料竄改或系統控制。對企業造成法規違規（如 GDPR、PCI-DSS）和聲譽損失。

B | 技術根因分析 (ROOT CAUSE & TECHNICAL ANALYSIS)

根因：API 端點未在伺服器端驗證調用者的角色權限，僅依賴前端隱藏按鈕或 UI 限制，未實施真正的後端授權檢查。

技術分析：掃描器使用低權限 token 成功呼叫應僅限管理員的端點，伺服器返回 200 OK 而非 403 Forbidden，確認後端缺少角色驗證邏輯。

偵測證據 (Evidence)：

Status: 200

C | AI 修復方案 (REMEDIATION STEPS)

AI

- 在每個敏感端點加入角色檢查 decorator
- 建立集中式權限中介層 (middleware)
- 採用 RBAC（角色存取控制）框架
- 定期進行授權邏輯的 Code Review
- 在 CI/CD 中加入自動化授權測試

D | 修復程式碼範例 (CODE EXAMPLE)

▸ Recommended Implementation

```
# FastAPI 範例：正確的角色檢查
from fastapi import Depends, HTTPException

def require_admin(current_user = Depends(get_current_user)):
    if current_user.role != 'admin':
        raise HTTPException(status_code=403, detail='Admin only')
    return current_user

@router.get('/admin/users')
def get_users(admin = Depends(require_admin)):
    return db.query(User).all()
```

E | 合規框架映射 (COMPLIANCE MAPPING)

OWASP API1:2023 (Broken Object Level Authorization) | PCI-DSS Req 6.4.1 (Protect against unauthorized access) | NIST SP 800-53 AC-3 (Access Enforcement)

GET /users/v1/_debug

A | 業務影響評估 (BUSINESS IMPACT)

攻擊者可以用普通用戶身份呼叫管理功能，導致帳號提權、資料竄改或系統控制。對企業造成法規違規（如 GDPR、PCI-DSS）和聲譽損失。

B | 技術根因分析 (ROOT CAUSE & TECHNICAL ANALYSIS)

根因：API 端點未在伺服器端驗證調用者的角色權限，僅依賴前端隱藏按鈕或 UI 限制，未實施真正的後端授權檢查。

技術分析：掃描器使用低權限 token 成功呼叫應僅限管理員的端點，伺服器返回 200 OK 而非 403 Forbidden，確認後端缺少角色驗證邏輯。

偵測證據 (Evidence)：

Status: 200

C | AI 修復方案 (REMEDIATION STEPS)

AI

- 在每個敏感端點加入角色檢查 decorator
- 建立集中式權限中介層 (middleware)
- 採用 RBAC（角色存取控制）框架
- 定期進行授權邏輯的 Code Review
- 在 CI/CD 中加入自動化授權測試

D | 修復程式碼範例 (CODE EXAMPLE)

▸ Recommended Implementation

```
# FastAPI 範例：正確的角色檢查
from fastapi import Depends, HTTPException

def require_admin(current_user = Depends(get_current_user)):
    if current_user.role != 'admin':
        raise HTTPException(status_code=403, detail='Admin only')
    return current_user

@router.get('/admin/users')
def get_users(admin = Depends(require_admin)):
    return db.query(User).all()
```

E | 合規框架映射 (COMPLIANCE MAPPING)

OWASP API1:2023 (Broken Object Level Authorization) | PCI-DSS Req 6.4.1 (Protect against unauthorized access) | NIST SP 800-53 AC-3 (Access Enforcement)

GET /users/v1

A | 業務影響評估 (BUSINESS IMPACT)

攻擊者可以用普通用戶身份呼叫管理功能，導致帳號提權、資料竄改或系統控制。對企業造成法規違規（如 GDPR、PCI-DSS）和聲譽損失。

B | 技術根因分析 (ROOT CAUSE & TECHNICAL ANALYSIS)

根因：API 端點未在伺服器端驗證調用者的角色權限，僅依賴前端隱藏按鈕或 UI 限制，未實施真正的後端授權檢查。

技術分析：掃描器使用低權限 token 成功呼叫應僅限管理員的端點，伺服器返回 200 OK 而非 403 Forbidden，確認後端缺少角色驗證邏輯。

偵測證據 (Evidence)：

Status: 200

C | AI 修復方案 (REMEDIATION STEPS)

AI

- 在每個敏感端點加入角色檢查 decorator
- 建立集中式權限中介層 (middleware)
- 採用 RBAC（角色存取控制）框架
- 定期進行授權邏輯的 Code Review
- 在 CI/CD 中加入自動化授權測試

D | 修復程式碼範例 (CODE EXAMPLE)

▸ Recommended Implementation

```
# FastAPI 範例：正確的角色檢查
from fastapi import Depends, HTTPException

def require_admin(current_user = Depends(get_current_user)):
    if current_user.role != 'admin':
        raise HTTPException(status_code=403, detail='Admin only')
    return current_user

@router.get('/admin/users')
def get_users(admin = Depends(require_admin)):
    return db.query(User).all()
```

E | 合規框架映射 (COMPLIANCE MAPPING)

OWASP API1:2023 (Broken Object Level Authorization) | PCI-DSS Req 6.4.1 (Protect against unauthorized access) | NIST SP 800-53 AC-3 (Access Enforcement)

GET /users/v1/_debug

A | 業務影響評估 (BUSINESS IMPACT)

攻擊者可以用普通用戶身份呼叫管理功能，導致帳號提權、資料竄改或系統控制。對企業造成法規違規（如 GDPR、PCI-DSS）和聲譽損失。

B | 技術根因分析 (ROOT CAUSE & TECHNICAL ANALYSIS)

根因：API 端點未在伺服器端驗證調用者的角色權限，僅依賴前端隱藏按鈕或 UI 限制，未實施真正的後端授權檢查。

技術分析：掃描器使用低權限 token 成功呼叫應僅限管理員的端點，伺服器返回 200 OK 而非 403 Forbidden，確認後端缺少角色驗證邏輯。

偵測證據 (Evidence)：

Status: 200

C | AI 修復方案 (REMEDIATION STEPS)

AI

- 在每個敏感端點加入角色檢查 decorator
- 建立集中式權限中介層 (middleware)
- 採用 RBAC（角色存取控制）框架
- 定期進行授權邏輯的 Code Review
- 在 CI/CD 中加入自動化授權測試

D | 修復程式碼範例 (CODE EXAMPLE)

▸ Recommended Implementation

```
# FastAPI 範例：正確的角色檢查
from fastapi import Depends, HTTPException

def require_admin(current_user = Depends(get_current_user)):
    if current_user.role != 'admin':
        raise HTTPException(status_code=403, detail='Admin only')
    return current_user

@router.get('/admin/users')
def get_users(admin = Depends(require_admin)):
    return db.query(User).all()
```

E | 合規框架映射 (COMPLIANCE MAPPING)

OWASP API1:2023 (Broken Object Level Authorization) | PCI-DSS Req 6.4.1 (Protect against unauthorized access) | NIST SP 800-53 AC-3 (Access Enforcement)

GET /createdb

A | 業務影響評估 (BUSINESS IMPACT)

攻擊者可以用普通用戶身份呼叫管理功能，導致帳號提權、資料竄改或系統控制。對企業造成法規違規（如 GDPR、PCI-DSS）和聲譽損失。

B | 技術根因分析 (ROOT CAUSE & TECHNICAL ANALYSIS)

根因：API 端點未在伺服器端驗證調用者的角色權限，僅依賴前端隱藏按鈕或 UI 限制，未實施真正的後端授權檢查。

技術分析：掃描器使用低權限 token 成功呼叫應僅限管理員的端點，伺服器返回 200 OK 而非 403 Forbidden，確認後端缺少角色驗證邏輯。

偵測證據 (Evidence)：

Status: 200

C | AI 修復方案 (REMEDIATION STEPS)

AI

- 在每個敏感端點加入角色檢查 decorator
- 建立集中式權限中介層 (middleware)
- 採用 RBAC（角色存取控制）框架
- 定期進行授權邏輯的 Code Review
- 在 CI/CD 中加入自動化授權測試

D | 修復程式碼範例 (CODE EXAMPLE)

▸ Recommended Implementation

```
# FastAPI 範例：正確的角色檢查
from fastapi import Depends, HTTPException

def require_admin(current_user = Depends(get_current_user)):
    if current_user.role != 'admin':
        raise HTTPException(status_code=403, detail='Admin only')
    return current_user

@router.get('/admin/users')
def get_users(admin = Depends(require_admin)):
    return db.query(User).all()
```

E | 合規框架映射 (COMPLIANCE MAPPING)

OWASP API1:2023 (Broken Object Level Authorization) | PCI-DSS Req 6.4.1 (Protect against unauthorized access) | NIST SP 800-53 AC-3 (Access Enforcement)

GET /createdb

A | 業務影響評估 (BUSINESS IMPACT)

攻擊者可以用普通用戶身份呼叫管理功能，導致帳號提權、資料竄改或系統控制。對企業造成法規違規（如 GDPR、PCI-DSS）和聲譽損失。

B | 技術根因分析 (ROOT CAUSE & TECHNICAL ANALYSIS)

根因：API 端點未在伺服器端驗證調用者的角色權限，僅依賴前端隱藏按鈕或 UI 限制，未實施真正的後端授權檢查。

技術分析：掃描器使用低權限 token 成功呼叫應僅限管理員的端點，伺服器返回 200 OK 而非 403 Forbidden，確認後端缺少角色驗證邏輯。

偵測證據 (Evidence)：

Status: 200

C | AI 修復方案 (REMEDIATION STEPS)

AI

- 在每個敏感端點加入角色檢查 decorator
- 建立集中式權限中介層 (middleware)
- 採用 RBAC（角色存取控制）框架
- 定期進行授權邏輯的 Code Review
- 在 CI/CD 中加入自動化授權測試

D | 修復程式碼範例 (CODE EXAMPLE)

▸ Recommended Implementation

```
# FastAPI 範例：正確的角色檢查
from fastapi import Depends, HTTPException

def require_admin(current_user = Depends(get_current_user)):
    if current_user.role != 'admin':
        raise HTTPException(status_code=403, detail='Admin only')
    return current_user

@router.get('/admin/users')
def get_users(admin = Depends(require_admin)):
    return db.query(User).all()
```

E | 合規框架映射 (COMPLIANCE MAPPING)

OWASP API1:2023 (Broken Object Level Authorization) | PCI-DSS Req 6.4.1 (Protect against unauthorized access) | NIST SP 800-53 AC-3 (Access Enforcement)

GET /users/v1/name1

A | 業務影響評估 (BUSINESS IMPACT)

攻擊者可以遍歷 ID 存取任意用戶的資料，導致大規模個資洩漏，違反 GDPR 等法規，面臨高額罰款。

B | 技術根因分析 (ROOT CAUSE & TECHNICAL ANALYSIS)

根因：API 在處理請求時，只驗證用戶是否已登錄，未驗證請求中的資源 ID 是否屬於該用戶。

技術分析：攻擊者只需修改 URL 中的物件 ID（如 /users/123 改為 /users/124）即可存取他人資料，伺服器無任何阻攔。

偵測證據 (Evidence)：

Status: 200

C | AI 修復方案 (REMEDIATION STEPS) AI

- 在每次資料查詢時加入 owner 驗證
- 使用 UUID 替代自增 ID
- 建立資源擁有者中介層
- 記錄所有跨用戶存取嘗試
- 定期滲透測試驗證修復效果

D | 修復程式碼範例 (CODE EXAMPLE)

▸ Recommended Implementation

```
# 正確做法：查詢時加入 user_id 過濾
@router.get('/items/{item_id}')
def get_item(item_id: int, current_user = Depends(get_current_user), db = Depends(get_db)):
    item = db.query(Item).filter(
        Item.id == item_id,
        Item.owner_id == current_user.id # 關鍵：驗證擁有者
    ).first()
    if not item:
        raise HTTPException(404, 'Not found')
    return item
```

E | 合規框架映射 (COMPLIANCE MAPPING)

OWASP API1:2023 | GDPR Article 25 (Data Protection by Design) | ISO 27001 A.9.4 (Access Control)

GET /users/v1/admin

A | 業務影響評估 (BUSINESS IMPACT)

攻擊者可以遍歷 ID 存取任意用戶的資料，導致大規模個資洩漏，違反 GDPR 等法規，面臨高額罰款。

B | 技術根因分析 (ROOT CAUSE & TECHNICAL ANALYSIS)

根因：API 在處理請求時，只驗證用戶是否已登錄，未驗證請求中的資源 ID 是否屬於該用戶。

技術分析：攻擊者只需修改 URL 中的物件 ID（如 /users/123 改為 /users/124）即可存取他人資料，伺服器無任何阻攔。

偵測證據 (Evidence)：

Status: 200

C | AI 修復方案 (REMEDIATION STEPS) AI

- 在每次資料查詢時加入 owner 驗證
- 使用 UUID 替代自增 ID
- 建立資源擁有者中介層
- 記錄所有跨用戶存取嘗試
- 定期滲透測試驗證修復效果

D | 修復程式碼範例 (CODE EXAMPLE)

▸ Recommended Implementation

```
# 正確做法：查詢時加入 user_id 過濾
@router.get('/items/{item_id}')
def get_item(item_id: int, current_user = Depends(get_current_user), db = Depends(get_db)):
    item = db.query(Item).filter(
        Item.id == item_id,
        Item.owner_id == current_user.id # 關鍵：驗證擁有者
    ).first()
    if not item:
        raise HTTPException(404, 'Not found')
    return item
```

E | 合規框架映射 (COMPLIANCE MAPPING)

OWASP API1:2023 | GDPR Article 25 (Data Protection by Design) | ISO 27001 A.9.4 (Access Control)

3. 合規性聲明 (Compliance Statement)

本報告之稽核基準及修復建議參照以下國際安全框架：

框架	適用範圍	關聯發現類型
OWASP API Security Top 10 (2023)	Web API 通用漏洞	BFLA, BOLA, Data Exposure
GDPR Article 25 / 32	個人資料保護	Data Exposure, Missing Headers
PCI-DSS v4.0 Req 6.4	支付卡資料安全	BFLA, Authentication
NIST SP 800-53 Rev.5	存取控制 (AC) 系列	Authorization failures
ISO/IEC 27001:2022 A.8.4	存取資訊和資產	All High/Medium findings

本報告由 AI API Security Scanner 自動生成，AI 分析輔助由 OpenAI GPT-3.5-turbo 提供。本報告僅作為技術參考，最終修復方案應由具資格的安全工程師審核確認。